RAIN FALL PREDICTION SYSTEM USING MACHINE LEARNING AND FLASK

ABSTRACT

Rainfall prediction plays a crucial role in agriculture, disaster management, and climate monitoring. This project presents a Rainfall Prediction System developed using Machine Learning and Flask framework.
The system predicts whether rainfall will occur based on weather attributes such as minimum temperature,
maximum temperature, and previous rainfall data.

A Random Forest classifier is trained using historical weather datasets.
The trained model is integrated into a Flask web application that allows users to input weather values
and receive instant rainfall predictions. The system performs preprocessing, feature selection, model training, evaluation, and deployment efficiently.

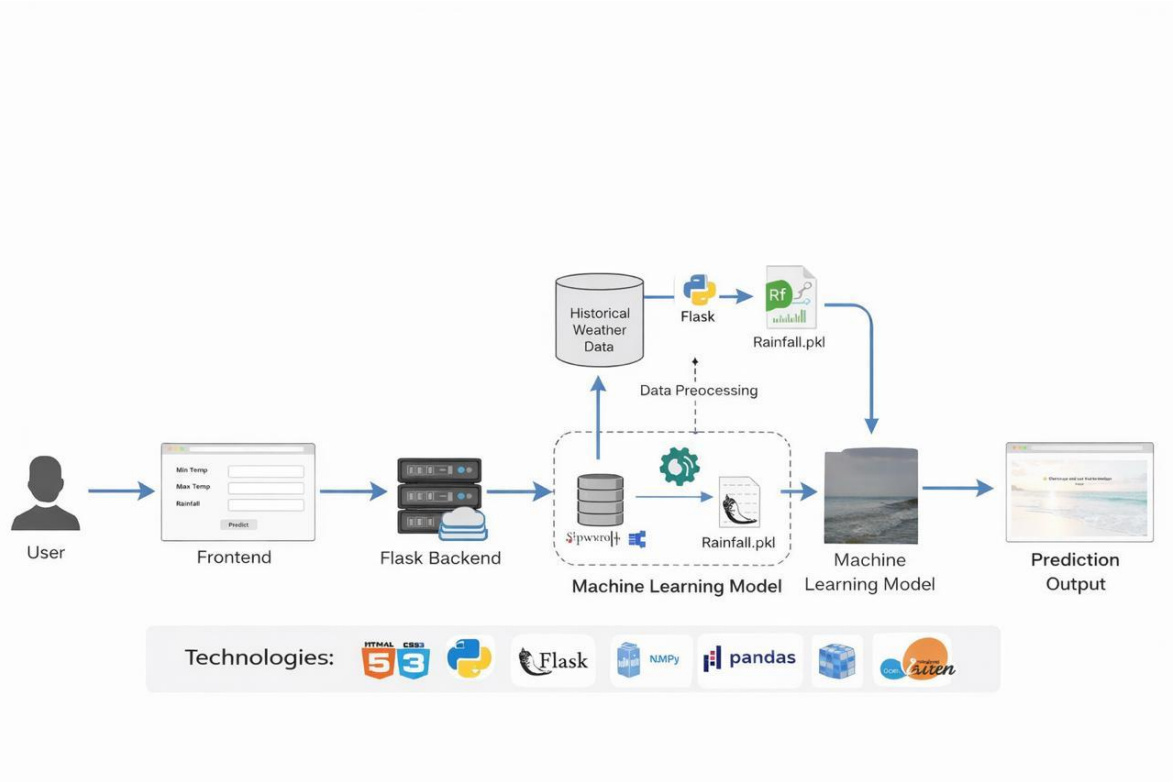
SYSTEM ARCHITECTURE

The technical architecture follows a layered structure:

1. Presentation Layer – HTML/CSS user interface.
2. Application Layer – Flask backend handling routing and requests.
3. Machine Learning Layer – Random Forest trained model.
4. Data Layer – Historical weather dataset.

Workflow Explanation:
User inputs weather parameters → Flask receives HTTP POST request →
Input validation and preprocessing → Data passed to trained model →
Model returns classification result → Flask renders result page.

This architecture ensures modularity, scalability, and maintainability.



Phase 1: Brainstorming Phase

This initial phase focused on identifying the problem domain and understanding real-world challenges
associated with rainfall unpredictability. The team analyzed agricultural losses, water resource mismanagement,
and sudden climate variations.

Multiple predictive approaches were studied including Linear Regression, Logistic Regression, Decision Trees, Support Vector Machines, and Ensemble Learning techniques.
After comparison based on accuracy, robustness, and overfitting resistance,
Random Forest was chosen.

Feasibility analysis was conducted covering technical feasibility, economic feasibility,
and operational feasibility. Expected system outcomes were clearly documented.

Phase 2: Requirement Analysis Phase

In this phase, requirements were systematically gathered and categorized.

Functional Requirements:
- Accept weather input parameters.
- Validate and preprocess inputs.
- Generate rainfall prediction.
- Display result clearly on user interface.

Non-Functional Requirements:
- Response time under 2 seconds.
- Prediction accuracy above 80%.
- Secure and reliable backend processing.
- Maintainable and scalable design.

Constraints and assumptions were also defined clearly.

Phase 3: Project Design Phase

The design phase translated requirements into technical blueprints.

System design included:
- Data Flow Diagram (DFD)
- Component Diagram
- Architecture Diagram

Database schema was analyzed for structured dataset usage.
Feature engineering decisions were finalized.
UI wireframes were created for user interaction simplicity.

Model design included selecting target variable (RainTomorrow)
and feature variables (temperature, humidity, rainfall).

Phase 4: Project Planning Phase

A structured timeline was created using milestone-based planning.

Planning breakdown:
- Week 1: Data Collection & Cleaning
- Week 2: Exploratory Data Analysis
- Week 3: Model Development
- Week 4: Backend Integration
- Week 5: Testing & Debugging
- Week 6: Documentation & Review

Risk Management:
- Data imbalance handled using sampling.
- Overfitting reduced through hyperparameter tuning.
- Integration errors minimized through modular testing.

Phase 5: Project Development Phase

Implementation activities were executed in this phase.

Data preprocessing steps:
- Handling missing values
- Encoding categorical variables
- Normalizing numerical features

Model Development:
- Train-test split (80:20 ratio)
- Random Forest training
- Hyperparameter tuning
- Accuracy evaluation using confusion matrix

Deployment:
- Model serialized using Pickle
- Flask routes implemented
- HTML templates connected.

Phase 6: Documentation Phase

Comprehensive documentation was prepared covering:

- Introduction
- Problem Statement
- Literature Review
- Methodology
- System Architecture
- Implementation Details
- Testing Results
- Future Enhancements

Screenshots of outputs and architecture diagrams were included.
Code explanation was documented for maintainability.

Phase 7: Project Demonstration Phase

The final system was demonstrated in a structured presentation.

Demonstration included:
- Explanation of project objective
- Architecture walkthrough
- Live model execution
- Real-time prediction display

Performance metrics such as accuracy and precision were explained.
Future improvements like real-time API integration and cloud deployment were discussed.

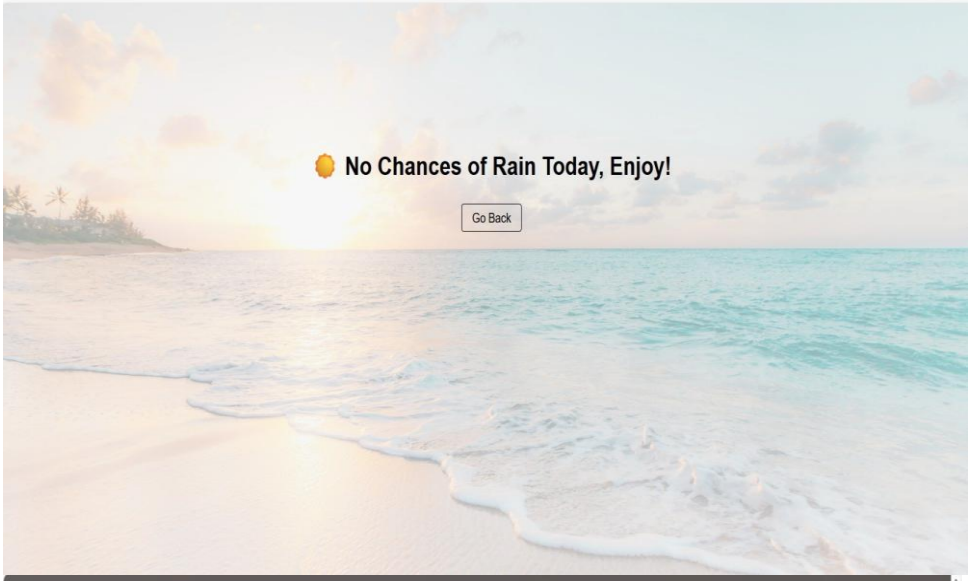
RESULTS AND OUTPUT

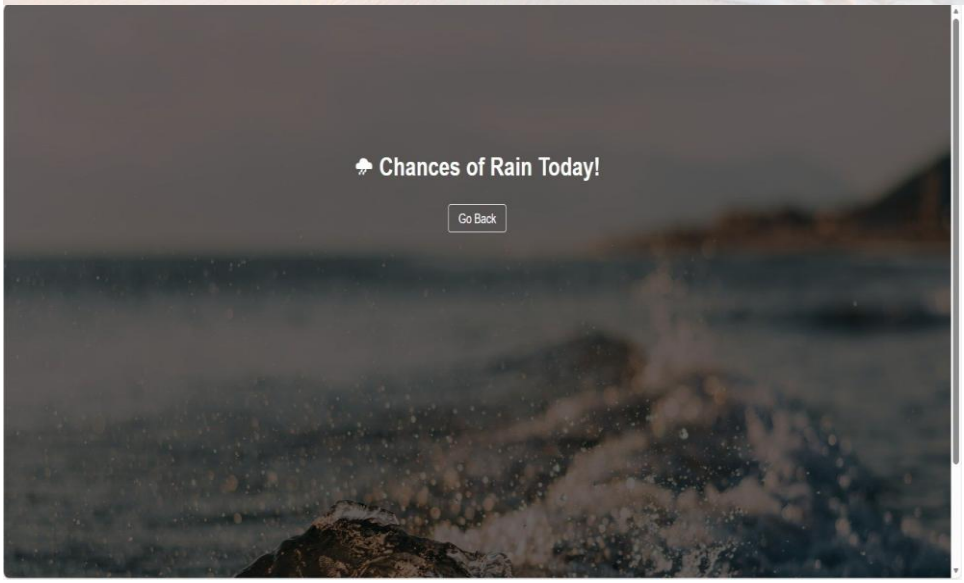
The model achieved strong prediction accuracy on test data.
Sample outputs:

Input: MinTemp=20, MaxTemp=32, Rainfall=5mm
Output: Chances of Rain Today

Input: MinTemp=25, MaxTemp=38, Rainfall=0mm
Output: No Chances of Rain Today

The system responds instantly and demonstrates stable performance.





CONCLUSION

The Rainfall Prediction System successfully integrates Machine Learning and Web Development.
The Random Forest model provides reliable predictions.
This project demonstrates practical implementation of data science
for solving real-world environmental challenges.